НИЯУ МИФИ

Сессии задач и планировщик

DOC-06.2

Платформа для изучения и практики курса по криптографии

Криптотренажёр mephi-12

Версия 0.9

Редакция от 07.11.2026

Роль	ФИО	Подпись / Дата
Разработал	Анохин Г.А.	
Согласовано		
Утверждено		

Москва, 2026

Лист регистрации изменений

Версия	Дата	Автор	Содержание изменения	Утвердил
0.9	07.11.2026	Анохин Г.А.	Уточнены правила и контракты; расширено описание сценариев; подготовлена печатная редакция.	

Содержание

Лист регистрации изменений **1**

1 Назначение и состав **4**

1.1 Данные и их жизненный цикл 4

2 Создание сессии **5**

2.1 SWR-SES-001. Получение пользователя 5

2.2 SWR-SES-002. Выбор доступной конфигурации 5

2.3 SWR-SES-003. Защита от двух активных сессий 6

2.4 SWR-SES-004. Формирование задач 6

2.5 SWR-SES-005. Сохранение созданного набора 6

2.6 SWR-SES-006. Сохранение времени создания 7

3 Обработка ответа **7**

3.1 SWR-SES-007. Регистрация времени поступления 7

3.2 SWR-SES-008. Проверка допуска 8

3.3 SWR-SES-009. Проверка формата 8

3.4 SWR-SES-010. Передача профильной проверке 9

3.5 SWR-SES-011. Сохранение вердикта 9

3.6 SWR-SES-012. Согласование одновременных обращений 9

3.7 SWR-SES-013. Сохранение итогового состояния 10

3.8 SWR-SES-014. Отказ без расхода попытки 10

3.9 SWR-SES-015. Завершение своевременной проверки 11

4 Контроль срока и итог **11**

4.1 SWR-SES-016. Фоновый контроль 11

4.2 SWR-SES-017. Завершение по просрочке 12

4.3 SWR-SES-018. Общая функция итога сессии 12

4.4 SWR-SES-019. Сохранность данных 12

4.5 SWR-SES-020. Предел обработки 13

4.6 SWR-SES-021. Запись принятой операции 13

4.7 SWR-SES-022. Форматы и категории API 14

4.8 SWR-SES-023. Чтение сессии и результата 14

5 Временные границы **15**

5.1 Согласование параллельных событий 15

5.2 Переходы состояния задачи 16

5.3 Проверка длительной обработки 16

6 Трассировка и описательные проверки **17**

7 Полнота описания подсистемы **18**

8 DOC-06.2. Контракты сессий задач **18**

8.1 Операции 18

8.2 Представление сессии и задачи 19

8.3 Предметный ответ 19

8.4 Результат и отказы 20
8.5 Как клиент проходит операции 21
8.6 Учебный пример решения Эль-Гамаля 21
8.7 Смысл связей в передаваемых данных 22
8.8 Коды и представление ошибок 22
8.9 Принятая обработка и восстановление 23
8.10 Чтение после потери ответа 24

Поле	Значение
Версия	0.9
Дата	07.11.2026
Ответственный	Анохин Г.А.
Основания	Разделы сессий и интеграции DOC-05; DOC-06
Плановый срок	06.11.2026

1 Назначение и состав

Документ определяет программные обязанности при создании сессии, обработке ответа, учёте срока и завершении задач. Он раскрывает системные требования сессий, требования интеграции и общие программные требования.

Логическая часть	Ответственность
Приём пользовательской операции	Получить подтверждённого пользователя, параметры и время поступления ответа
Создание сессии	Проверить конфигурацию и ограничение активных сессий, сформировать и сохранить набор
Обработка ответа	Проверить допуск, вызвать предметную проверку, сохранить результат
Контроль срока	Завершить просроченные задачи, учитывая своевременные ответы в обработке
Вычисление итога	Определить состояние сессии по состояниям её задач
Хранение	Согласовать изменения и сохранить данные для последующих операций

Это распределение обязанностей; оно не задаёт имена классов, таблиц или число процессов. Для внешних участников используется контракт операций. Внутренняя последовательность и согласование параллельных действий определены ниже.

1.1 Данные и их жизненный цикл

Сессия связывает владельца, время создания и состав задач. Для каждой задачи сохраняются принадлежность сессии, условие, тип и состояние. После выдачи набора условие и тип используются при проверке того же задания; они не восстанавливаются заново из текущей конфигурации группы.

Обработка ответа использует дополнительные сведения об операции: момент получения запроса и факт его незавершённой обработки. Эти сведения нужны для согласования с контролем срока и не вводят новое публичное состояние задачи. Пока предметный итог не определён, задача остаётся `NEW`.

Окончание обработки и изменение предметного состояния — связанные события. Вердикт завершает попытку и определяет состояние задачи. Ошибка формата и технический сбой без вердикта прекращают текущую обработку без расхода попытки; после срока к незавершён-

ной задаче применяется правило просрочки. Поэтому снятие отметки об обработке также должно учитываться контролем срока.

2 Создание сессии

Приоритет всех программных требований документа — обязательный. Источники SR указывают на определения в DOC-05. Описания проверок находятся в разделе 6.

2.1 SWR-SES-001. Получение пользователя

Обработчик сессий начинает работу с контекста доступа. Из него он получает пользователя, от имени которого выполняются создание и отправка ответа. Параметры операции описывают выбранную конфигурацию или задачу, но не определяют владельца. Это применение общего контракта передачи личности внутри подсистемы.

Требование. Обработчик создания и отправки ответа должен получать пользователя через компонент проверки доступа до обращения к его сессиям.

Далее при создании ищется группа этого пользователя, при ответе проверяется владелец сохранённой сессии. Неуспешная проверка доступа прекращает операцию до предметного вычисления. Пользовательский ID, присланный в теле, не заменяет контекст доступа. Внешние условия определены в контракте операций, а проверка принадлежности задачи — в порядке допуска ответа.

Источники: SR-SES-003, SR-SYS-001.

2.2 SWR-SES-002. Выбор доступной конфигурации

Имя конфигурации выбирает состав нового набора в пределах группы пользователя. Поиск по всему каталогу без учёта группы мог бы выдать недоступное задание. Поэтому управление сессиями сначала получает группу, а затем включённую для неё конфигурацию. Системное назначение такой границы дано в согласованном составе набора.

Требование. Создание сессии должно получать состав выбранной конфигурации по группе подтверждённого пользователя и отклонять запрос при отсутствии группы или включённой конфигурации с указанным именем.

Отсутствие группы и отсутствие включённой конфигурации прекращают создание до выдачи задач. Клиент получает `CONFIGURATION_NOT_FOUND`, поскольку для него нет доступного состава с указанным именем. После успешного выбора используются полученные данные конфигурации, а не повторные параметры клиента. Дальнейшее формирование описано в генерации задач, формат параметра — в контракте создания.

Источники: SR-SES-001, SR-SYS-003.

2.3 SWR-SES-003. Защита от двух активных сессий

Два запроса одного пользователя могут одновременно пройти чтение «активной сессии нет». Если после этого каждый независимо создаст набор, будет нарушено системное ограничение. Поэтому проверка и создание рассматриваются как одна операция под блокировкой пользователя. Параллельность клиентских запросов не должна менять правило одного незавершённого набора.

Требование. Проверка отсутствия сессии `NEW` и создание новой выполняются в транзакции под блокировкой строки пользователя, не допускающей две активные сессии при одновременных запросах.

Первый запрос создаёт сессию, второй после получения доступа к данным видит уже существующий набор и получает отказ. Перед решением учитывается завершение старой просроченной сессии, но набор с обрабатываемым своевременным ответом остаётся незавершённым. Общая логика завершения описана в вычислении итога, категория отказа — в таблице ошибок API.

Источник: SR-SES-002. Перед проверкой учитывается завершение просроченной сессии. Сессия с обрабатываемым своевременным ответом остаётся незавершённой.

2.4 SWR-SES-004. Формирование задач

Выбранная конфигурация становится конкретными заданиями через генераторы соответствующих типов. Для каждого задания сохраняются условие, тип и связь с создаваемой сессией. Эти данные нужны и для выдачи пользователю, и для будущего вызова проверяющего. Одно только сохранение имени конфигурации не заменяет сохранения выданного условия.

Требование. Создание должно получать условия у обработчиков типов, заданных выбранной конфигурацией, и сохранять состав сессии вместе с условиями, типами и принадлежностью задач.

Если набор содержит ранцевую задачу и Эль-Гамаля, к каждой относится свой генератор и полученное им условие. После выдачи эти условия не генерируются заново при проверке. Соответствие типов задано в контракте предметного обработчика, а согласованное сохранение полного состава — в следующем требовании. Неподдерживаемый тип не заменяется другим заданием.

Источники: SR-SES-001, SR-SYS-002, SR-SYS-003, SR-SYS-004. Последующее изменение конфигурации не должно подменять выданные условия.

2.5 SWR-SES-005. Сохранение созданного набора

Пользователь должен получить полный набор по выбранной конфигурации. Частично сохранённая сессия после отказа одного генератора мешала бы повторить создание и не соответствовала бы обещанному составу. Поэтому операция имеет общую границу подтверждения: либо сохранён весь набор, либо создание не считается завершённым.

Требование. Компонент создания должен подтверждать операцию только после согласованного сохранения полного набора. Ошибка генерации или сохранения не должна оставлять пользователю частично созданную активную сессию.

Ошибка генерации или записи откатывает создание сессии и её задач. На внешней границе возвращается технический отказ; ограничение активных сессий не должно блокировать пользователя из-за оставшегося частичного набора. Это частный случай согласованного сохранения. Для уже созданной сессии применяется другой путь — фиксация результата отдельного ответа.

Источники: SR-SES-001, SR-SYS-005. Сохранение набора выполняется одной транзакцией PostgreSQL; ошибка откатывает создание сессии и её задач.

2.6 SWR-SES-006. Сохранение времени создания

От времени создания начинается общий интервал решения. Значение должно относиться к сессии и сохраняться вместе с ней, чтобы обработчик ответа, планировщик и восстановление использовали одну основу. Повторное чтение не означает нового создания, а перезапуск приложения не означает начала нового учебного набора.

Требование. При создании набора ПО должно зафиксировать серверное время создания сессии и использовать его для общего срока `D`, определённого в карточке домена. Время не изменяется при чтении, ошибках, перезапуске или повторном обращении.

Общий срок вычисляется по сохранённому `t0`; отдельное время создания каждой записи задачи не должно предоставлять ей другой интервал. Момент поступления ответа хранится отдельно и сравнивается с этим сроком. Связь временных величин приведена в карточке домена, а действия планировщика — в завершении по просрочке.

Источники: SR-SES-006, SR-SYS-006, SR-SYS-007.

3 Обработка ответа

3.1 SWR-SES-007. Регистрация времени поступления

Задача обработчика приёма — сохранить момент получения полного ответа до ожидания очереди и предметного вычисления. Иначе задержка внутри сервера могла бы превратить вовремя отправленное решение в позднее. Используется серверная отметка, связанная с конкретной операцией; время клиента не задаёт право на допуск.

Требование. ПО должно фиксировать время получения полного запроса сервером до ожидания предметной проверки и сохранять связь этой отметки с обрабатываемым ответом. Время запуска проверяющего не должно подменять время поступления.

Зафиксированное `t_in` сопровождает запрос при ожидании и восстановлении. По нему проверяются общий срок сессии и предел обработки. Сведения о принятой операции должны быть доступны контролю просрочки, чтобы он не закрыл задачу во время допустимой провер-

ки. Хранение операции описано в записи принятого ответа, системное основание — в правиле своевременности.

Источники: SR-SES-011, SR-SYS-006. Сведения о своевременном запросе должны учитываться при согласовании с контролем срока. Отметка клиента не используется как основание допуска.

3.2 SWR-SES-008. Проверка допуска

До вычисления необходимо установить, какой задаче адресован ответ и допустима ли работа с ней. Проверки связывают подтверждённого пользователя, сохранённую принадлежность и состояния задачи и сессии. Только после этого можно интерпретировать тело решения по нужному предметному типу. UUID сам по себе права доступа не даёт.

Требование. Обработка ответа должна проверять наличие задачи, принадлежность сессии текущему пользователю, состояния задачи и сессии и время поступления запроса до вызова предметного проверяющего.

Для чужой задачи используется отказ без раскрытия её данных. Для собственной задачи учитывается также поступление запроса относительно срока и наличие уже принятой обработки. Формат ответа не должен выбираться по произвольному клиентскому типу до установления задачи. Категории отказов приведены в контракте ошибок, следующая стадия — в проверке формата.

Источники: SR-SES-003, SR-SES-008, SR-SES-011. Для своевременного ответа учитывается его зарегистрированная обработка; фоновая просрочка не должна преждевременно лишать его допуска.

3.3 SWR-SES-009. Проверка формата

Транспортный разбор и предметное решение отвечают на разные вопросы. Сначала устанавливается, содержит ли запрос все обязательные поля нужных типов. Затем математический проверяющий определяет правильность расчёта. Неверный JSON нельзя считать неправильным решением задачи: такой запрос ещё не содержит допустимого предметного ответа.

Требование. ПО должно проверять формат и обязательные поля ответа по схеме сохранённого типа задачи до предметного вердикта. Синтаксическую ошибку нельзя передавать как неверное решение.

Для ранцевого решения и Эль-Гамаля используются разные структуры из предметного контракта. Ошибка синтаксиса, отсутствующее поле или неверный тип дают `INVALID_REQUEST` без расхода попытки. Корректные по форме значения с неверными математическими связями передаются предметной проверке. Результаты этих стадий различаются по общему правилу категории операции.

Источники: SR-SES-009, SR-SYS-009. Набор и типы обязательных полей определены в

контракте сессий.

3.4 SWR-SES-010. Передача профильной проверке

После допуска и проверки формата управление сессиями вызывает предметный обработчик. Выбор делается по сохранённому типу задачи, а проверка получает её сохранённое условие. Подмена этих данных значениями из пользовательского ответа изменила бы само задание. Обязательство соответствует связи генерации и проверки.

Требование. ПО должно передавать проверяющему сохранённое условие выбранной задачи и полученный ответ, определяя обработчик по сохранённому типу задачи.

Домен сессий не повторяет математические шаги профильного проверяющего и не выводит вердикт из текста исключения. Он получает явный предметный результат либо технический отказ. При отсутствии нужного обработчика операция не переводится в неверное решение. Дальнейший путь положительного или отрицательного вердикта определён в сохранении результата.

Источники: SR-SES-004, SR-SYS-004. Неизвестный тип или недоступный обработчик должны приводить к техническому отказу без предметного вердикта.

3.5 SWR-SES-011. Сохранение вердикта

Вердикт вычисления нужно превратить в сохранённый результат задачи. В этой же операции учитывается состояние остальных задач сессии и закрывается запись обработки. Пока фиксация не завершена, внешний ответ не должен подтверждать результат как записанный. Такой порядок связывает вычисление с общим контрактом сохранения.

Требование. После завершённой предметной проверки ПО должно сохранить `SOLVED` при вердикте «верно» или `FAILED` при вердикте «неверно», вычислить соответствующее состояние сессии и только затем подтвердить результат операции.

Перед записью проверяется, что операция ещё допускает фиксацию: её предел времени не истёк и итог не был установлен другим завершением. В короткой транзакции сохраняются результат задачи и пересчитанное состояние набора. После потери соединения эти данные доступны через чтение сессии, без нового вызова проверяющего и повторной предметной попытки.

Источники: SR-SES-005, SR-SES-007, SR-SYS-005. Результат содержит идентификатор и тип проверенной задачи по контракту.

3.6 SWR-SES-012. Согласование одновременных обращений

Параллельные обращения создают риск двух результатов для одной задачи или неверного итога для всего набора. Для одной задачи нужно исключить две независимые фиксации. Для разных задач одной сессии нужно согласовать пересчёт агрегированного состояния. Здесь

общая граница — данные сессии, а не отдельное клиентское соединение.

Требование. ПО должно согласовывать обращения к одной задаче так, чтобы одновременно полученные ответы не приводили к двум завершённым предметным проверкам или перезаписи итогового результата.

Предметное вычисление не удерживает долгую транзакцию. При фиксации строки блокируются в установленном порядке: сессия, задача, запись операции. После получения блокировки решение принимается по актуальным данным. Повтор для занятой задачи получает `CHECK_IN_PROGRESS`. Сценарии разных порядков событий описаны в разборе параллельных событий, а защита итогового состояния — в правиле повторного ответа.

Источник: SR-SES-008. Для фиксации используется короткая транзакция с блокировкой сначала строки сессии, затем задачи и записи операции; предметное вычисление выполняется вне этой транзакции. Повторное обращение к занятой задаче возвращает `CHECK_IN_PROGRESS`. При параллельном завершении разных задач итог вычисляется по согласованному набору их состояний под блокировкой сессии.

3.7 SWR-SES-013. Сохранение итогового состояния

После сохранения `SOLVED` или `FAILED` задача имеет итог, который нельзя пересмотреть новой отправкой решения. Это распространяется на повторное нажатие, повтор сетевого клиента и отдельное новое соединение. Сервер принимает решение по сохранённым данным, поэтому состояние интерфейса пользователя не влияет на запрет второй попытки.

Требование. Для задачи в `SOLVED` или `FAILED` обработчик должен отклонять новый ответ, сохраняя существующий результат без повторной предметной проверки.

Новый ответ отклоняется до профильного вычисления; существующий вердикт не заменяется и не записывается повторно как новая попытка. Для восстановления отображения используется операция чтения результата. Если запрос получен после общего срока, приоритет категории отказа определяется контрактом ошибок; ни один из отказов не изменяет прежний итог.

Источники: SR-SES-008, SR-SYS-005.

3.8 SWR-SES-014. Отказ без расхода попытки

Отказ без предметного вердикта не должен расходовать возможность решения. Это правило относится к ошибке формата и техническому сбою проверки, но не отменяет общий срок сессии. Текущая обработка завершается соответствующей категорией, после чего возможность нового запроса оценивается по состоянию задачи и времени.

Требование. При ошибке формата или техническом сбое без вердикта ПО должно сообщать соответствующую категорию отказа и сохранять неиспользованную попытку. До срока задача остаётся `NEW`, если не завершена по другому основанию.

До `D` задача сохраняет `NEW`, если она не завершена по другому основанию. Новый исправ-

ленный или повторный запрос имеет собственное время поступления. После D незавершённая задача подлежит просрочке, даже если предметная попытка не была израсходована. Системный смысл разницы дан в обработке технического сбоя, а предел текущей операции — в ограничении обработки.

Источники: SR-SES-009, SR-SES-010, SR-SYS-009. После срока отсутствие допустимой незавершённой проверки приводит к завершению по просрочке; отказ не продлевает срок.

3.9 SWR-SES-015. Завершение своевременной проверки

Общий срок прекращает приём новых ответов, но уже полученный допустимый ответ может ещё находиться в обработке. Если фоновый компонент в этот момент назначит задаче просрочку, студент потеряет своевременно использованную возможность решения. Поэтому обработчик и планировщик должны учитывать одну запись принятой операции.

Требование. ПО должно принимать вердикт допустимого ответа с `t_in < D`, даже если проверка закончилась при `t >= D`. Регистрация и завершение такой проверки должны быть согласованы с фоновым контролем срока.

Для `t_in < D` проверка может зафиксировать вердикт после D, пока не истёк её собственный предел. Отказ в допуске, ошибка формата или техническое завершение снимают защиту от просрочки. Если общий срок к этому времени уже прошёл, задача закрывается по времени. Связанные действия определены в контроле просрочки и пределе обработки.

Источники: SR-SES-011, SR-SYS-006. Пока своевременный запрос проходит проверки допуска и формата либо предметную проверку, его задаче нельзя назначать итог по просрочке. Если запрос отклонён или проверка завершилась техническим сбоем после срока, защита от просрочки прекращается; новый ответ после срока не допускается.

4 Контроль срока и итог

4.1 SWR-SES-016. Фоновый контроль

Фоновый контроль обслуживает задачи независимо от действий пользователя. Он нужен, чтобы незавершённые наборы получили итог и без нового запроса. Его период не должен становиться частью разрешённого студенту интервала: между D и следующим запуском поздние ответы уже недопустимы.

Требование. ПО должно выполнять фоновую обработку просроченных задач с плановым периодом 60 секунд и отдельно проверять срок при допуске каждого нового ответа.

Обработчик каждого ответа самостоятельно применяет временное правило к `t_in`. Планировщик тем временем приводит сохранённые состояния к тому же правилу. Поэтому задержка фонового запуска влияет на момент обслуживания записи, но не выдаёт дополнительное время решения. Порядок выбора задач дан в завершении по просрочке, общий контракт — в едином правиле времени.

Источники: SR-SES-006, SR-SYS-006; период задан [TS.REQ.FUNC.SESSION.TIME]. Период фоновой обработки не предоставляет пользователю дополнительное время между сроком и очередным запуском.

4.2 SWR-SES-017. Завершение по просрочке

Назначение просрочки затрагивает только незавершённые задачи без своевременного запроса в обработке. Наличие старого NEW в хранилище ещё не означает, что можно записать FAILED: нужно учесть общий срок и принятую операцию. Уже завершённый вердикт не пересматривается планировщиком.

Требование. При обработке просрочки ПО должно переводить в FAILED только задачи NEW, срок которых наступил и для которых нет своевременного запроса в обработке. Итоговые результаты остальных задач должны сохраняться.

После выбора подходящих задач сохраняются их новые состояния и пересчитываются сессии. Для задачи со своевременной обработкой планировщик ждёт её исхода в пределах установленного времени. Если операция закончилась отказом после D, задача снова подпадает под правило просрочки. Само вычисление итога одинаково для всех путей и описано в общей функции состояния сессии.

Источники: SR-SES-006, SR-SES-011. После отказа незавершённой проверки за пределами срока задача подлежит этому же правилу.

4.3 SWR-SES-018. Общая функция итога сессии

Обработчик ответа и планировщик могут завершать разные задачи одного набора. Они не должны иметь разные способы определения успешности сессии. Итог выводится из полного согласованного набора состояний задач, а не из типа события, которое привело к последней записи.

Требование. Обработка ответа и контроль срока должны применять одно правило: пока есть задача без итогового состояния, сессия NEW; когда все задачи завершены, сессия SOLVED, только если все задачи SOLVED, иначе FAILED.

Если есть хотя бы одна задача без итога, сессия остаётся NEW. После завершения всех задач наличие хотя бы одной FAILED означает неуспех набора, даже когда последняя задача решена верно. Только все SOLVED дают успешную сессию. Системная формулировка находится в итоге сессии, а защита пересчёта при одновременных ответах — в согласовании обращений.

Источники: SR-SES-007, SR-SYS-005, SR-SYS-006.

4.4 SWR-SES-019. Сохранность данных

Сессия должна сохранять смысл после остановки процесса: тот же владелец, тот же набор условий и прежние временные границы. Одного итогового флага для этого недостаточно.

При чтении и проверке нужны идентификаторы, принадлежность, тип и сохранённое условие каждой задачи.

Требование. Хранение должно сохранять идентификаторы, принадлежность, состав, условия, серверное время создания и определённые состояния сессии и задач между запусками приложения.

После восстановления данные не генерируются заново и срок не отсчитывается от запуска приложения. Завершённые результаты выдаются чтением. Для операции, не дошедшей до вердикта, используются отдельные сведения о принятом ответе. Системная цель закреплена в сохранности данных, программные пути — в восстановлении операции и чтении сессии.

Источники: SR-SYS-002, SR-SYS-007. Принятые ответы сохраняются до вычисления и восстанавливаются по SWR-SES-021.

4.5 SWR-SES-020. Предел обработки

Обработка своевременного ответа имеет конечный предел. В него входит не только математическое вычисление, но и ожидание очереди и продолжение после сбоя. Этот предел нужен, чтобы операция не оставалась бессрочно принятой и не удерживала задачу от завершения по общему сроку.

Требование. ПО должно ограничивать всю обработку принятого ответа сроком `t_in + 60 секунд`. Результат может быть зафиксирован только до этого момента. По достижении срока незавершённая операция получает `CHECK_TIMEOUT`; её поздний результат отбрасывается без изменения задачи.

Перед фиксацией проверяется и состояние операции, и её срок. После `t_in + 60 секунд` результат уже не записывается, даже если вычисление завершилось позднее. Техническое завершение не создаёт отрицательный предметный вердикт. Дальнейшее состояние задачи определяется правилом отказа без расхода попытки и правилом просрочки.

Источники: SR-SES-010, SR-SES-012. Ожидание очереди, вычисление и восстановление входят в один срок; повторный запуск его не продлевает. После отказа применяется общее правило срока сессии.

4.6 SWR-SES-021. Запись принятой операции

Запись принятой операции связывает ответ пользователя с задачей и временем его получения. Она нужна, потому что память процесса и исходное сетевое соединение могут исчезнуть раньше фиксации результата. Для восстановления должны остаться сами входные данные и прежний предел обработки, а не только отметка «что-то выполнялось».

Требование. До предметной проверки ПО должно сохранить запись операции с UUID, владельцем, задачей, ответом, `t_in`, сроком обработки и признаком незавершённости. Фиксация вердикта и завершение записи операции выполняются в одной транзакции с изменением состояний задачи и сессии.

При перезапуске сохранённая незавершённая операция продолжается в оставшемся интервале. Готовый результат и закрытие записи фиксируются вместе с состояниями задачи и сессии. Если результат уже записан, повторная обработка не создаёт новой попытки. Если срок операции истёк, применяется техническое завершение по пределу. Пользователь получает готовые данные через чтение собственной сессии.

Источники: SR-SES-008, SR-SES-013, SR-SYS-005, SR-SYS-007. При запуске приложения незавершённые операции с неистёкшим сроком продолжаются по сохранённым данным. Получение записи обработчиком согласуется блокировкой, исключающей две одновременно выполняемые обработки. Уже зафиксированный результат не создаёт новой предметной попытки. Операции с истёкшим сроком завершаются по SWR-SES-020.

4.7 SWR-SES-022. Форматы и категории API

Внешний контракт превращает системные категории в точные параметры, поля и HTTP-ответы. Разбор запроса должен следовать ему до обращения к предметному алгоритму. Иначе разные клиенты могли бы получать разный смысл одного и того же запроса, а ошибки формата смешивались бы с неверным решением.

Требование. Приём операций должен применять обязательные поля, представление времени и целых чисел, предел тела и соответствие ошибок HTTP-кодам из контракта сессий. Повторяющийся параметр конфигурации отклоняется до создания набора.

Повторяющееся имя конфигурации не выбирается произвольно из нескольких значений: запрос отклоняется. Большие целые передаются установленными строками без потери точности, время — в UTC, а размер тела ограничивается до предметной проверки. Подробности находятся в операциях, схемах решения и машинных кодах ошибок. Эти правила не изменяют математический критерий правильности.

Источники: SR-SES-009, SR-SYS-008, SR-SYS-009. Ошибки формата и превышения размера не запускают предметную проверку. Неизвестный тип в сохранённой конфигурации даёт технический отказ без частичной сессии.

4.8 SWR-SES-023. Чтение сессии и результата

Клиенту нужен самостоятельный способ получить уже сохранённое состояние. После потери ответа он не должен повторно решать задачу или вызывать проверяющий, чтобы узнать итог. Чтение по ID сессии проверяет владельца и возвращает те данные, которые уже связаны с этим набором в хранилище.

Требование. ПО должно предоставлять `GET /api/tasks/session/{sessionId}` для чтения собственной сессии с состояниями задач и сохранёнными предметными результатами. При отсутствии сессии у текущего пользователя возвращается `SESSION_NOT_FOUND`.

Операция доступна и после общего срока. Она показывает состояния задач и сохранённый `result`, а для просроченной задачи без предметного вердикта — `null` вместо выдуманного

решения. Перед выдачей учитывается правило просрочки с защитой своевременных обработок. Назначение описано в системном требовании чтения, поля ответа — в контракте восстановления отображения.

Источники: SR-SES-014, SR-SYS-002, SR-SYS-007, SR-SYS-008. Чтение не расходует попытку и доступно после общего срока. Перед формированием ответа состояние просроченных задач приводится к правилу SWR-SES-017 с сохранением своевременных обработок. Сохранённый вердикт возвращается без повторного вычисления.

5 Временные границы

Обозначения и длительность заданы в карточке домена.

Ситуация	Ожидаемое действие
Запрос получен до `D`, проверка завершена до `D`	Сохранить предметный результат
Запрос получен до `D`, проверка завершена после `D`, но до `t_in + 60 секунд`	Сохранить предметный результат; не заменять его просрочкой
Запрос получен точно в `D` или позже	Отклонить новый ответ по сроку
Своевременный запрос оказался некорректным до `D`	Сообщить ошибку; исправленный новый запрос возможен до `D`
Некорректный запрос или технический сбой обнаружены после `D`	Сообщить категорию отказа; завершить незавершённую задачу по просрочке, если нет другой допустимой проверки
Планировщик запущен во время своевременной проверки	Не назначать этой задаче `FAILED` по времени
Планировщик задержался	Отклонять новые поздние ответы независимо от состояния фоновой обработки

5.1 Согласование параллельных событий

Конкурировать могут два ответа на одну задачу, ответы на разные задачи одной сессии, ответ и контроль просрочки, а также два запроса создания набора. В каждом случае требуется сохранить определённый системой результат. Выбор блокировок или другого механизма не должен изменять правило одной попытки и общего срока.

Для одной задачи окончательный предметный вердикт определяется не более одного раза. Пока первый допустимый ответ обрабатывается, другое обращение не должно независимо завершать вторую предметную проверку. Если первый запрос закончился без вердикта до срока, возможность нового допустимого обращения сохраняется по общим правилам.

При параллельной проверке разных задач согласование необходимо на уровне сессии. Завершение последней задачи должно учитывать уже определённые результаты остальных. Сохранение одного вердикта не должно перезаписывать итог, вычисленный без учёта другой за-

вершившейся задачи.

При конкуренции ответа и планировщика существенен момент поступления запроса. Своевременный запрос сохраняет право пройти проверки допуска и формата и, если они успешны, получить предметный вердикт. Ответ, поступивший после срока, этого права не получает даже при ещё не обновлённом состоянии задачи в хранилище.

5.2 Переходы состояния задачи

Исходное состояние и событие	Следующее состояние	Значение для попытки
`NEW`, допустимый ответ признан верным	`SOLVED`	Предметная попытка завершена
`NEW`, допустимый ответ признан неверным	`FAILED`	Предметная попытка завершена
`NEW`, ошибка формата до срока	`NEW`, если нет другого основания завершения	Попытка сохраняется
`NEW`, технический сбой без вердикта до срока	`NEW`, если нет другого основания завершения	Попытка сохраняется
`NEW`, наступил срок, своевременного запроса в обработке нет	`FAILED`	Завершение по сроку, без нового предметного вердикта
`NEW`, срок наступил во время своевременной проверки	`NEW` до завершения обработки	Ожидание единственного результата
`SOLVED` или `FAILED`, новый ответ	Прежнее итоговое состояние	Повторная проверка не допускается

Для сессии используется агрегирование состояний задач по SWR-SES-018. Программная реализация не должна считать любой набор завершённых задач успешным: наличие хотя бы одной `FAILED` определяет итог `FAILED`. Это правило одинаково после предметной проверки и после фоновой обработки.

5.3 Проверка длительной обработки

Для требования своевременности нужно рассматривать получение запроса, запуск проверяющего, наступление срока и сохранение вердикта как отдельные события. Условие `t_in < D` проверяется по первому из них. Задержка между событиями не изменяет сохранённую отметку времени и не разрешает вторую попытку.

Отдельно рассматривается своевременная обработка, завершившаяся отказом после срока. Предметный результат в этом случае отсутствует, но срок для нового запроса уже закончился. Требования сохраняют различие причин: технический сбой не становится неверным решением, а последующее завершение задачи происходит по просрочке.

6 Трассировка и описательные проверки

Ниже перечислены связи всех программных требований документа. Это описания проверки требований, не результаты прогонов и не выпущенные кейсы DOC-10.

SWR	Системные источники	Условия и ожидаемый результат проверки
SWR-SES-001	SR-SES-003, SR-SYS-001	Пользователь запроса определяется проверкой доступа, чужой ID его не заменяет
SWR-SES-002	SR-SES-001, SR-SYS-003	Без группы или доступной конфигурации набор не создаётся
SWR-SES-003	SR-SES-002	Одновременное создание не оставляет две активные сессии
SWR-SES-004	SR-SES-001, SR-SYS-002, SR-SYS-003, SR-SYS-004	Состав, условия и связи соответствуют выбранной конфигурации
SWR-SES-005	SR-SES-001, SR-SYS-005	Сбой генерации не оставляет частично созданный набор
SWR-SES-006	SR-SES-006, SR-SYS-006, SR-SYS-007	Время создания и общий срок не изменяются при повторе и восстановлении
SWR-SES-007	SR-SES-011, SR-SYS-006	Ожидание обработчика не меняет время поступления
SWR-SES-008	SR-SES-003, SR-SES-008, SR-SES-011	Чужая, завершённая или несвоевременная задача не передаётся проверяющему
SWR-SES-009	SR-SES-009, SR-SYS-009	Ошибка формата отклоняется без предметного вердикта
SWR-SES-010	SR-SES-004, SR-SYS-004	Обработчик и условие соответствуют сохранённому типу
SWR-SES-011	SR-SES-005, SR-SES-007, SR-SYS-005	Вердикт, состояние задачи и итог сессии сохраняются согласованно
SWR-SES-012	SR-SES-008	Два параллельных ответа не создают два предметных итога
SWR-SES-013	SR-SES-008, SR-SYS-005	Повтор после итогового состояния сохраняет результат
SWR-SES-014	SR-SES-009, SR-SES-010, SR-SYS-009	Отказ без вердикта не расходует попытку; срок сохраняется
SWR-SES-015	SR-SES-011, SR-SYS-006	Вердикт своевременного ответа принимается после срока
SWR-SES-016	SR-SES-006, SR-SYS-006	Поздний ответ отклоняется и при задержке фоновой обработки
SWR-SES-017	SR-SES-006, SR-SES-011	Просрочка меняет только незавершённые задачи без своевременной обработки
SWR-SES-018	SR-SES-007, SR-SYS-005, SR-SYS-006	Любая задача `FAILED` даёт сессии `FAILED` после завершения всех задач
SWR-SES-019	SR-SYS-002, SR-SYS-007	После перезапуска принадлежность, срок и завершённые результаты сохранены
SWR-SES-020	SR-SES-010, SR-SES-012	По достижении срока обработки поздний результат не фиксируется
SWR-SES-021	SR-SES-008, SR-SES-013, SR-SYS-005, SR-SYS-007	Перезапуск сохраняет время и входные данные операции; итог фиксируется один раз
SWR-SES-022	SR-SES-009, SR-SYS-008, SR-SYS-009	Неоднозначные параметры и неверные форматы отклоняются установленной категорией

SWR	Системные источники	Условия и ожидаемый результат проверки
SWR-SES-023	SR-SES-014, SR-SYS-002, SR-SYS-007, SR-SYS-008	После потери ответа чтение сессии возвращает сохранённый вердикт без новой попытки

7 Полнота описания подсистемы

Контракт задаёт создание, отправку ответа и чтение сессии, входные данные, результаты и категории отказов. Программные требования охватывают создание, предметную проверку, ограничение времени, параллельные обращения, сохранение и восстановление. Предел обработки и фоновый период имеют разные назначения, хотя оба составляют 60 секунд: первый ограничивает одну операцию, второй задаёт частоту фонового контроля общего срока.

История: 05.11.2026 — подготовлена версия 0.8. 07.11.2026 — расширены сценарии взаимодействия; определены формат API, предел обработки и восстановление принятого ответа; добавлены LaTeX и PDF.

Редакция 07.11.2026: развёрнуты пояснения каждого требования, добавлены примеры и точные ссылки на связанные разделы. Формулировки обязательств и идентификаторы сохранены.

8 DOC-06.2. Контракты сессий задач

Приложение к DOC-06.2, версия 0.9, редакция от 07.11.2026. Ответственный: Анохин Г.А. Здесь определены операции, форматы данных, категории отказов и правила восстановления домена сессий.

8.1 Операции

Операция	Вход	Нормальный результат
`POST /api/tasks/session`	Параметр строки запроса `configuration`: имя конфигурации	`201 Created`, созданная сессия с задачами
`PUT /api/tasks/submit`	Параметр строки запроса `statementId`: UUID задачи; тело JSON с решением её типа	`200 OK`, вердикт с идентификатором и типом задачи
`GET /api/tasks/session/{sessionId}`	UUID собственной сессии в пути	`200 OK`, состояние сессии и сохранённые результаты задач

Операции выполняются от имени подтверждённого пользователя с ролью `USER`. Идентификатор владельца не берётся из тела запроса. Время получения полного запроса сервером используется по SWR-SES-007; предметный допуск определяется в DOC-06.2. JSON передаётся в UTF-8 с типом содержимого `application/json`.

Создание принимает ровно один непустой параметр `configuration`. Имя сопоставляется с конфигурацией группы с учётом регистра. Повторяющийся параметр отклоняется с `400 Bad Request`; несколько наборов одним запросом не создаются. Для отправки ответа требуется ровно один корректный UUID `statementId`.

Предельный размер тела ответа — 1 МиБ, то есть 1 048 576 байт до разбора JSON. Превышение даёт `413 Payload Too Large` и не расходует предметную попытку. Отсутствующее тело, некорректный JSON, неверный тип поля и отсутствие обязательного поля дают `400 Bad Request`.

8.2 Представление сессии и задачи

Поле сессии	Содержание
`id`	UUID сессии
`createdAt`	Серверное время создания
`state`	`NEW`, `SOLVED` или `FAILED`
`userId`	UUID владельца
`problems`	Список задач сессии

Поле задачи	Содержание
`id`	UUID задачи; передаётся в `statementId` при отправке ответа
`statement`	Строка, содержащая JSON условия соответствующего типа
`type`	Тип, определяющий генератор и проверяющий
`state`	Состояние задачи
`sessionId`	UUID содержащей её сессии
`result`	Сохранённый предметный результат либо `null`, если вердикт не зафиксирован

Выдаваемые идентификаторы и принадлежность должны соответствовать сохранённым данным. Время передаётся строкой UTC в формате RFC 3339 с миллисекундами и суффиксом `Z`. Поле `createdAt` задаёт начало общего срока. Сравнение со сроком выполняется на сервере; часовой пояс устройства пользователя не участвует в допуске ответа.

8.3 Предметный ответ

Структура тела определяется сохранённым типом задачи. Транспортный разбор JSON предшествует предметной проверке. Значение поля `type` в решении не может переключить проверку на другой тип задачи.

В сессиях поддерживаются `BACKPACK` и `EL_GAMAL`. Для каждого типа назначены генератор и проверяющий с форматом ответа ниже. Наличие другого типа в конфигурации считается ошибкой настройки: создание набора завершается техническим отказом, частичная сессия не сохраняется.

8.3.1 Ранцевая задача

Все перечисленные поля обязательны и не допускают `null`. `power` передаётся положительным целым JSON-числом, `type` — строкой варианта, совпадающей с вариантом сохранённого условия. `message` и `decodedMessage` — массивы JSON-значений `true` и `false`. Остальные целые значения передаются десятичными строками; поля ранцев — массивами таких строк.

Поле	Предметное значение
`power`	Размерность задачи
`type`	Обозначение варианта ранцевой задачи
`message`	Исходное сообщение как список логических значений
`lightBackpack`	Лёгкий ранец, список целых чисел
`omega`	Множитель преобразования
`hardBackpack`	Тяжёлый ранец, список целых чисел
`encodedMessage`	Зашифрованное сообщение
`decodedMessage`	Восстановленное сообщение как список логических значений
`module`	Модуль преобразования
`reverseOmega`	Обратный множитель

8.3.2 Эль-Гамаль

Все поля решения обязательны, не допускают `null` и передаются десятичными строками:

Поле	Предметное значение
`y`	Открытый ключ
`k`	Одноразовый параметр шифрования
`c1`	Первая часть шифротекста
`c2`	Вторая часть шифротекста
`decryptedMessage`	Восстановленное сообщение

Десятичная строка содержит `0` либо ненулевое целое без ведущих нулей, перед которым допускается знак минуса. Пробелы, знак плюса, дробная часть и экспоненциальная запись не допускаются. Такой формат сохраняет точность больших целых на стороне клиента. Правильность математических связей между значениями определяет предметная проверка; синтаксически корректный, но неверный расчёт даёт предметный вердикт `isOk = false`.

8.4 Результат и отказы

Поле предметного результата	Содержание
`problemId`	UUID проверенной задачи
`isOk`	`true` при верном решении, `false` при неверном
`problemType`	Сохранённый тип задачи

Поле предметного результата	Содержание
`errorMessage`	Непустая строка при неверном решении; `null` при верном

`isOk = false` означает завершённую проверку неправильного решения. Отсутствие прав, ошибка формата, просрочка, недоступность проверяющего и ошибка сохранения передаются отдельными категориями отказов. При верном ответе `errorMessage` имеет значение `null`; при неверном содержит пояснение предметной ошибки.

Состояние задачи и попытки определяется требованиями DOC-06.2, а не формулировкой диагностического сообщения. При техническом сбое нельзя сообщать, что решение неверно, если предметного вердикта не было.

8.5 Как клиент проходит операции

Создание набора начинается с имени конфигурации. Ответ содержит сессию и список задач; клиент сохраняет её `id`, чтобы позднее читать тот же набор. Каждое задание имеет собственный `id`, который используется как `statementId` при отправке решения. Отождествлять эти два идентификатора нельзя: первый обозначает набор, второй — выбранную задачу внутри него.

Клиент выбирает схему тела по выданному типу задачи. Он передаёт предметные значения в установленном формате, а сервер берёт условие из сохранённых данных. Правильность формы запроса проверяется до математического вердикта. Поэтому `400 INVALID_REQUEST` означает, что решение не было допущено к предметной проверке, а `200 OK` с `isOk = false` означает завершённую неверную попытку.

После получения результата клиент может обновить отображение через чтение сессии. Этот же путь используется при потере соединения: неизвестность на стороне клиента не доказывает, что сервер не сохранил вердикт. Повторная отправка решения не заменяет чтение, поскольку задача могла уже завершиться. Подробный порядок раскрывает чтение после потери ответа.

8.6 Учебный пример решения Эль-Гамаля

Пусть выданное условие задаёт `p = 23`, `g = 5`, `x = 6`, `m = 13`. Для выбранного в решении `k = 3` получаются `y = 8`, `c1 = 10`, `c2 = 9`, а восстановленное сообщение равно `13`. Числа служат примером чтения формата; условия конкретной задачи берутся из её сохранённого `statement`.

Тело ответа на эту задачу имеет вид:

```
{
  "y": "8",
  "k": "3",
  "c1": "10",
```

```
  "c2": "9",
  "decryptedMessage": "13"
}
```

Числа заключены в кавычки, поскольку контракт использует десятичные строки для целых произвольной длины. Поле `isOk` в результате, напротив, имеет логический тип. Если эти значения признаны верными, сохраняется `SOLVED`; неверный расчёт в корректно оформленном теле приводит к `FAILED` с отрицательным предметным результатом. Отличие формы и смысла ответа описано в проверке формата и передаче профильному проверяющему.

8.7 Смысл связей в передаваемых данных

`statementId` обозначает идентификатор задачи, полученный в поле `id` этой задачи при создании набора. Это не текст условия и не отдельный идентификатор, который клиент должен назначать самостоятельно. Сервер устанавливает принадлежность по сохранённой задаче и её сессии.

Поле `sessionId` связывает задачу с набором, а `userId` в представлении сессии обозначает владельца. Наличие этих значений в выдаче не меняет источник полномочий: операция выполняется от имени подтверждённого пользователя. Сервер сопоставляет переданный идентификатор задачи со своими данными принадлежности.

У `statement` и тела решения разное назначение. Первое передаёт выданное условие; второе содержит ответ пользователя по предметной схеме. Проверка использует сохранённое условие, поэтому клиентский ответ не должен заменять его другим заданием. Конкретный состав решения подтверждается владельцем соответствующего типа.

Для результата проверки идентификатор и тип позволяют сопоставить вердикт с задачей. Диагностический текст поясняет неверное решение; при отсутствии профильного пояснения возвращается «Решение неверно». Он не должен служить единственным признаком категории отказа: нарушение доступа и техническая ошибка отличаются от предметного `isOk = false` независимо от языка сообщения.

8.8 Коды и представление ошибок

HTTP	Код ошибки	Условие
`400`	`INVALID_REQUEST`	Неверные параметры, JSON, типы или обязательные поля
`401`	`UNAUTHENTICATED`	Нет действительных данных аутентификации
`403`	`ACCESS_DENIED`	У пользователя нет роли `USER`
`404`	`TASK_NOT_FOUND`	Задача не найдена среди задач текущего пользователя
`404`	`SESSION_NOT_FOUND`	Сессия не найдена среди сессий текущего пользователя
`404`	`CONFIGURATION_NOT_FOUND`	У группы нет включённой конфигурации с таким именем

HTTP	Код ошибки	Условие
409	ACTIVE_SESSION_EXISTS	Уже есть незавершённая сессия
409	TASK_FINALIZED	Задача или сессия уже имеет итоговое состояние
409	CHECK_IN_PROGRESS	Для задачи уже обрабатывается допустимый ответ
410	SESSION_EXPIRED	Новый ответ получен в момент общего срока или позднее
413	PAYLOAD_TOO_LARGE	Тело ответа превышает 1 МиБ
500	INTERNAL_ERROR	Ошибка настройки или сохранения данных
503	CHECK_UNAVAILABLE	Проверяющий недоступен либо произошёл технический сбой проверки
503	CHECK_TIMEOUT	Обработка не завершилась за 60 секунд

Для чужой задачи используется та же ошибка TASK_NOT_FOUND, что и для отсутствующей: интерфейс не раскрывает существование чужих заданий. Отсутствие группы при создании набора даёт CONFIGURATION_NOT_FOUND, поскольку доступная конфигурация не может быть выбрана.

Тело ошибки содержит строковые поля code, message, timestamp и requestId. timestamp передаётся в том же UTC-формате, что и createdAt; requestId — UUID серверной операции. Поле message предназначено для человека, а клиент определяет категорию по code. При ACTIVE_SESSION_EXISTS дополнительное поле details.sessionId содержит UUID собственной активной сессии; по нему клиент может восстановить её данные, если ответ на создание был потерян. Текст исключения, стек вызовов и служебные данные хранилища в ответ не включаются.

Проверки доступа предшествуют раскрытию данных задачи. После установления собственной задачи проверяется время: при t_in >= D возвращается SESSION_EXPIRED независимо от фонового обновления состояния. Для своевременного повторного ответа на уже завершённую задачу или сессию возвращается TASK_FINALIZED. Ни одна из этих ошибок не запускает вторую предметную попытку.

8.9 Принятая обработка и восстановление

Перед запуском проверяющего сохраняются UUID операции, пользователь, задача, полный ответ, t_in и срок обработки t_in + 60 секунд. Для одной задачи может существовать только одна незавершённая операция предметной проверки. Принятые данные не заменяются телом повторного запроса.

При перезапуске незавершённые операции продолжаются в пределах прежнего срока. Если результат уже сохранён, повторная фиксация не выполняется. Если срок обработки закончился, операция получает CHECK_TIMEOUT без нового предметного вердикта. В пределах общего срока пользователь может отправить новый ответ; после общего срока незавершённая задача закрывается по просрочке.

8.10 Чтение после потери ответа

Получение сессии возвращает сохранённые состояния и поле `result` каждой задачи. Для `SOLVED` оно содержит вердикт с `isOk = true`; для неверного решения — вердикт с `isOk = false`. У задачи, завершённой по просрочке без предметного вердикта, `result` равно `null`.

Клиент использует чтение, если соединение прервано до доставки ответа или требуется восстановить отображение после перезапуска. Чтение доступно после срока и не повторяет предметную проверку. Чужая сессия и отсутствующая сессия дают один ответ `SESSION_NOT_FOUND`.